

1- What are the benefits of using a generic sorting algorithm over a non-generic one?

A: Generic sorting provides type-safety and code reusability. It works with any data type without causing performance issues related to "Boxing and Unboxing" (converting value types to objects back and forth).

2- How do lambda expressions improve the readability and flexibility of sorting methods?

A: Lambda expressions (`=>`) allow you to pass the sorting logic directly inside the method call in a single, short line of code, eliminating the need to write separate, lengthy methods elsewhere.

3- Why is it important to use a dynamic comparer function when sorting objects of various data types?

A: Because complex objects (like `Employee`) can be sorted in multiple ways (by ID, Name, or Salary). A dynamic comparer delegate lets the developer inject the exact rule needed at runtime without changing the original sorting algorithm.

4- How does implementing `Comparable` in derived classes enable custom sorting?

A: It forces the derived class to define its own specific `CompareTo` logic (e.g., sorting `Manager` by Salary). This gives the class a "Natural Sort Order" that built-in sorting methods (like `Array.Sort()`) can use automatically.

5- What is the advantage of using built-in delegates like `Func<T, T, TResult>` in generic programming?

A: It saves time and standardizes the code. Instead of declaring custom delegates manually, built-in delegates like `Func`, `Action`, and `Predicate` are instantly recognizable by any C# developer.

6- How does the usage of anonymous functions differ from lambda expressions in terms of readability and efficiency?

A: Both compile to the same underlying IL code, so efficiency is identical. However, Lambda expressions are much shorter, cleaner, and easier to read, while anonymous functions require the clunky `delegate` keyword.

7- Why is the use of generic methods beneficial when creating utility functions like `Swap`?

A: A generic `Swap<T>` allows swapping integers, strings, or custom objects using a single piece of code. Without generics, you would have to write a separate `Swap` method for every single data type.

8- What are the challenges and benefits of implementing multi-criteria sorting logic in generic methods?

A: *Challenge:* The logic inside the delegate becomes slightly more complex.
Benefit: It provides extreme accuracy, allowing secondary and tertiary sorting (e.g., sorting by Salary, and if they are equal, sorting by Name alphabetically).

9- Why is the `default(T)` keyword crucial in generic programming, and how does it handle value and reference types differently?

A: When you don't know the exact type of `T`, `default(T)` safely assigns the correct default value without crashing. It returns `0` (or `false`) for value types, and `null` for reference types.

10- How do constraints in generic programming ensure type safety and improve the reliability of generic methods?

A: Constraints (like where `T : ICloneable`) inform the compiler exactly what capabilities `T` has. This prevents runtime errors by stopping developers from passing incompatible types that don't possess the required methods.

11- What are the benefits of using delegates for string transformations in a functional programming style?

A: It allows treating functions as "Data". You can create a pipeline of transformations (uppercase, reverse, substring) and pass them as parameters, leading to clean, modular, and functional code.

12- How does the use of delegates promote code reusability and flexibility in implementing mathematical operations?

A: It acts like the "Strategy Design Pattern". You write one main calculation engine, and inject the specific math operation (Add, Sub, Div) dynamically via a delegate depending on the user's need.

13- What are the advantages of using generic delegates in transforming data structures?

A: They allow for powerful mapping concepts (like LINQ's `.Select()`), where you can transform an entire list of `Employee` objects directly into a list of `Strings` (their names) flawlessly.

14- How does Func simplify the creation and usage of delegates in C#?

A: Func provides a pre-built signature that always returns a value (the last parameter type). It removes the boilerplate code needed to declare a custom delegate explicitly.

15- Why is Action preferred for operations that do not return values?

A: Action specifically indicates that the method performs a "Side Effect" (like logging, printing to console, or updating a database) and returns void, making the code's intent instantly clear.

16- What role do predicates play in functional programming, and how do they enhance code clarity?

A: Predicate is specifically designed for testing conditions. It takes an input and returns a bool. It makes filtering logic (like finding even numbers or active users) highly readable.

17- How do anonymous functions improve code modularity and customization?

A: They allow developers to define isolated, one-off logic right where it's needed without cluttering the class with helper methods that will only ever be used once.

18- When should you prefer anonymous functions over named methods in implementing mathematical operations?

A: When the operation is simple, brief, and exclusively relevant to that specific block of code. If the operation is complex or needs to be tested/reused across the system, a named method is better.

19- What makes lambda expressions an essential feature in modern C# programming?

A: Lambda expressions are the foundation of LINQ. They bring functional programming paradigms into C#, allowing developers to query, filter, and modify collections with unprecedented ease.

20- How do lambda expressions enhance the expressiveness of mathematical computations in C#?

A: They closely mimic actual mathematical notation (e.g., $f(x) = x^2$ is written as $x \Rightarrow x * x$), making the logic highly expressive and mathematically intuitive.

1- LinkedIn article about Delegates in implementing functional paradigm

Unlocking Functional Programming in C# with Delegates!

In C#, delegates are the bridge between Object-Oriented and Functional paradigms. They allow us to treat functions as first-class citizens—meaning we can assign methods to variables, pass them as arguments, and return them dynamically.

By utilizing built-in delegates like `Func<T>`, `Action<T>`, and `Predicate<T>`, alongside Lambda Expressions, developers can write incredibly concise, modular, and LINQ-friendly code. Stop writing massive loops and start treating behavior as data! #CSharp #DotNet #FunctionalProgramming #Delegates

2- Search Summaries:

Parallel Programming & Concurrency: This is about executing multiple tasks simultaneously to maximize multi-core CPU usage. In C#, tools like `Parallel.ForEach` split tasks into separate threads to speed up heavy computations.

Unit Testing & TDD: TDD (Test-Driven Development) is a workflow where developers write automated tests *before* writing the actual code. Unit Tests check individual methods in isolation to ensure logic holds up against edge cases.

Asynchronous Programming (async/await): (Detailed below in Part 3).